



FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ENGENHARIA DE SOFTWARE

JOVI STUDY LENS

BRUCE LI YAN TING RM: 569168

FELIPE RORATO COSTA RM: 570419

GIOVANNA ANDRADE PAREJAS RM: 569041

LAURA CARDIN MIRILLI RM: 570841

TURMA: 1ESPI

SPRINT 3 - COMPUTATIONAL THINKING WITH PYTHON

SÃO PAULO

2026

BRUCE LI YAN TING
FELIPE RORATO COSTA
GIOVANNA ANDRADE PAREJAS
LAURA CARDIN MIRILLI

SPRINT 3 - COMPUTATIONAL THINKING WITH PYTHON

Trabalho acadêmico apresentado ao Centro
Universitário FIAP como parte dos requisitos para
aprovação na disciplina de Computational Thinking
With Python do curso de Engenharia de Software.

Orientador: Patrícia Maura Angelini

SÃO PAULO

2026

SUMÁRIO

1. DESCRIÇÃO DO PROJETO.....	4
1.1 Objetivo geral.....	4
1.2 Justificativa.....	4
2. FUNCIONALIDADES IMPLEMENTADAS.....	5
2.1 Digitalizar Foto de Aula.....	5
2.2 Copiar conteúdo.....	5
2.3 Traduzir conteúdo.....	5
2.4 Biblioteca de materiais.....	5

1. DESCRIÇÃO DO PROJETO

O JOVI StudyLens é um protótipo em Python desenvolvido como parte do Challenge FIAP 2026, simulando uma funcionalidade do smartphone Jovi voltada ao estudante em tempo integral. O sistema transforma registros fotográficos feitos em sala de aula (lousas, slides, cadernos) em material de estudo organizado, pesquisável e reutilizável, acessível diretamente pelo terminal. Nesta Sprint 3, o projeto evoluiu com uma camada visual mais elaborada na interface de linha de comando, utilizando a biblioteca Rich, sem alteração nas regras de negócio já implementadas.

1.1 Objetivo geral

Eliminar a dor do estudante que acumula fotos de aula na galeria do celular sem organização ou utilidade prática, oferecendo um fluxo estruturado de captura, armazenamento, tradução e acesso ao conteúdo. Porém agora, apresentado em uma interface de terminal mais clara e agradável de usar

1.2 Justificativa

Estudantes frequentemente fotografam conteúdos em sala, mas essas imagens se perdem entre outras fotos no dispositivo, sem categorização por matéria, tipo ou data. O JOVI StudyLens resolve esse problema centralizando o conteúdo em uma biblioteca pessoal, com funcionalidades que vão além do simples armazenamento: o aluno pode copiar o texto extraído, traduzi-lo para outros idiomas e consultá-lo a qualquer momento de forma organizada. Na Sprint 3, além de resolver o problema funcional, buscou-se também melhorar a experiência de uso do protótipo em si, tornando a leitura das informações no terminal mais rápida e intuitiva por meio de cores, tabelas e painéis.

2. FUNCIONALIDADES IMPLEMENTADAS

2.1 Digitalizar Foto de Aula

Simula o processo de captura e extração de texto de um material fotográfico. O usuário informa o tipo de material (lousa, slide, caderno), define um título, seleciona a matéria correspondente e insere o conteúdo textual extraído, que em uma versão com IA seria gerado automaticamente via OCR. O material é então salvo na biblioteca com um ID único, permitindo recuperação posterior.

Alinhamento ao objetivo: reproduz a etapa de digitalização do fluxo Fotografia → IA processa → Aluno aprende, o núcleo da proposta do produto.

2.2 Copiar conteúdo

Permite ao usuário selecionar qualquer material salvo na biblioteca e copiar seu conteúdo textual diretamente para a área de transferência do sistema, utilizando a biblioteca pyperclip. O texto copiado pode ser colado em qualquer outro aplicativo, como editor de texto, e-mail, etc.

Alinhamento ao objetivo: elimina a necessidade de digitar repetidas vezes o conteúdo capturado, tornando o material imediatamente reutilizável.

2.3 Traduzir Conteúdo

Oferece tradução de textos do Português para o Inglês via API do Google Translator (utilizando a biblioteca deep-translator). O usuário pode traduzir um texto digitado manualmente ou selecionar diretamente um material da biblioteca. O resultado é exibido lado a lado com o original, registrado no histórico de traduções da sessão e, opcionalmente, salvo como novo material na biblioteca no idioma de destino.

Alinhamento ao objetivo: amplia o alcance do material de estudo, permitindo que o aluno utilize o conteúdo capturado em contextos bilíngues, tal como preparação para provas, leitura de artigos ou estudo de idiomas.

2.4 Biblioteca de materiais

Central de acesso a todos os materiais digitalizados na sessão, incluindo três exemplos pré-carregados (História, Filosofia e Matemática). Oferece cinco modos de navegação: visualização completa do acervo, filtragem por matéria, busca por palavra-chave no título ou

conteúdo, leitura integral de um material selecionado e resumo feito por IA para os materiais de exemplo. Nas opções de visualização completa, filtragem e busca, o sistema agora também oferece acesso rápido ao conteúdo: após exibir a lista, pergunta se o estudante deseja abrir algum material específico digitando o ID correspondente, restringindo a busca apenas aos itens daquela lista exibida.

Alinhamento ao objetivo: resolve diretamente a dor central do projeto, uma vez que o conteúdo fotografado deixa de estar perdido na galeria e passa a ser acessível de forma organizada, categorizada e pesquisável.

3. JUSTIFICATIVA DE ALTERAÇÕES REALIZADAS EM RELAÇÃO À SPRINT ANTERIOR

Nesta sprint, o projeto evoluiu em duas frentes: uma reformulação visual completa da interface de terminal, e um pequeno incremento funcional de navegabilidade na Biblioteca de Materiais. Nenhuma das quatro funcionalidades centrais (Digitalizar, Copiar, Traduzir e Biblioteca) foi removida ou teve sua regra de negócio original alterada.

Removido: Não houve remoção de funcionalidades, telas ou regras de negócio. A única remoção técnica foi a formatação de texto colorido feita manualmente via códigos ANSI (`"\033[31m"`), substituída pela abordagem descrita abaixo.

Modificado:

- Toda a exibição de texto no terminal, antes feita com `print()` simples, passou a utilizar a biblioteca Rich (`Console.print()`), permitindo cores, negrito e painéis.
- Os cabeçalhos de cada tela (função `titulo()`) passaram a ser exibidos como painéis (Panel) com borda e título destacado.
- A tabela de listagem de materiais (`exibir_lista_materiais()`) passou a utilizar o componente Table do Rich, com bordas e cabeçalho estilizado.
- Mensagens de erro e sucesso passaram a ser exibidas em vermelho/verde e negrito, respectivamente, facilitando a identificação visual.
- As opções numeradas dos menus (`[1]`, `[2]`, etc.) passaram a ser destacadas em ciano.

- A validação de entrada do ID no Resumo de um material (opção 5) foi corrigida: antes aceitava qualquer número positivo e travava o programa (TypeError) caso o ID não existisse; agora usa `max_val=3` no `input_validado()`, restringindo a entrada apenas aos três materiais de exemplo disponíveis para resumo, sem necessidade de tratamento de erro adicional.
- O fluxo de retorno ao menu foi ajustado para não pedir a confirmação de ENTER duas vezes seguidas: quando o estudante opta por não acessar nenhum material na nova função de acesso rápido (pressionando ENTER), o programa retorna direto ao menu principal, sem uma segunda solicitação redundante.

Adicionado:

- Dependência da biblioteca rich (rich==15.0.0), adicionada ao requirements.txt do projeto.
- Instância única de Console() no início do módulo, reutilizada por todas as funções de exibição.
- Função auxiliar `aviso_sair()`, para centralizar a repetição do aviso de saída.
- Função `exibir_conteudo_material()`, que centraliza a exibição do conteúdo completo de um material (reaproveitada tanto na leitura quanto no acesso rápido).
- Função `perguntar_acesso_material()`, nova funcionalidade que, após a exibição de uma lista de materiais (ver todos, filtrar por matéria ou buscar por palavra-chave), pergunta ao estudante se deseja abrir o conteúdo completo de algum item digitando seu ID, ou apenas pressionar ENTER para voltar ao menu. A busca é restrita à lista exibida no momento (evitando acessar materiais fora do filtro/busca aplicado) e insiste na mesma pergunta em caso de ID inválido, até receber uma entrada válida ou o ENTER de saída.